DATABASE 1 · LAB 04

Lab Tasks — From ERD to SQL

STUDENT	ACADEMIC NO.	DATE
Eyad Mohammed Elghonemy	132300082	19 Mar 2026

Part 1 — ERD Design

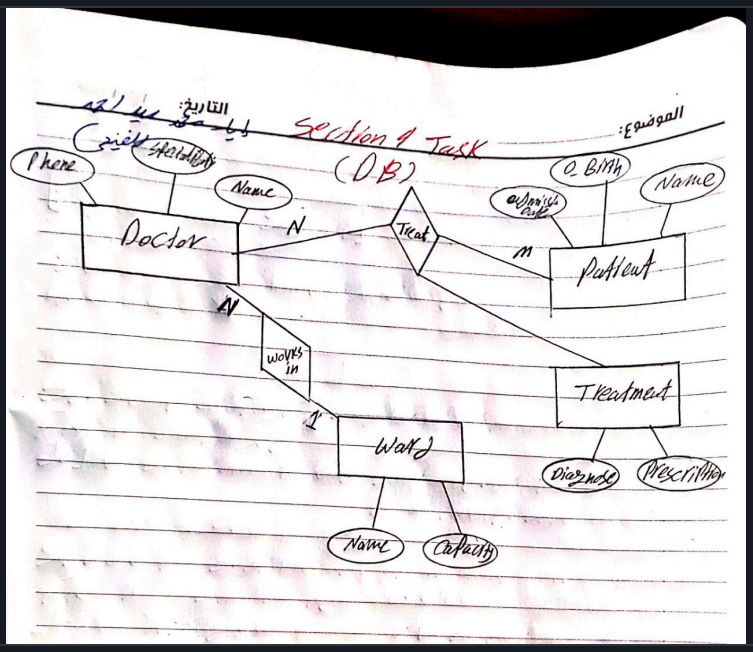
1 task · Draw the Hospital System ERD on paper and upload a photo

T 1.1 Read the **Hospital System** scenario below, then draw the complete **Chen notation ERD** on paper. Include all entities, attributes (with key attributes underlined), relationships, and cardinality labels.

> A hospital manages **doctors**, **patients**, and **wards**. Each doctor has a name, specialization, and phone number. Each patient has a name, date of birth, and admission date. Each ward has a name and capacity. A doctor works in one ward, but a ward can have many doctors. A doctor can treat multiple patients, and a patient can be treated by multiple doctors. Each treatment records the التشخيص والوصفة الطبية.

💡 Use the 7-step ERD design process: (1) Read → (2) Highlight nouns → (3) Highlight verbs → (4) List attributes → (5) Identify keys → (6) Determine cardinality → (7) Draw. You should find 3 strong entities + 1 junction entity.

PHOTO — YOUR HAND-DRAWN ERD



Part 2 — ERD → SQL Mapping

3 tasks · Convert your Hospital ERD into CREATE TABLE statements

T 2.1

Write **CREATE TABLE** statements for all 4 tables in the Hospital System: **Ward**, **Doctor**, **Patient**, and **Treatment** (junction). Include data types, PRIMARY KEY, FOREIGN KEY, and NOT NULL constraints where appropriate.

💡 Remember insert order matters: parent tables (Ward) first, then tables with FKs (Doctor), then junction tables (Treatment). Use INT for IDs, VARCHAR for text, DATE for dates.

YOUR SQL

```
CREATE TABLE Ward(

        WardID          INT             PRIMARY KEY,
        WardName        NVARCHAR(100),
        WardCapacity            INT

);

CREATE TABLE Doctor(

        DoctorID        INT             PRIMARY KEY,
        DoctorName  NVARCHAR(100)       ,
        DoctorSpecialization            NVARCHAR(150),
        DoctorPhone             NVARCHAR(20),
        WardID          INT             FOREIGN KEY     REFERENCES
Ward(WardID)
);

CREATE TABLE Patient(

        PatID           INT             PRIMARY KEY,
        PatName         NVARCHAR(100)   NOT NULL,
        PatBirthDate            DATE,
);

CREATE TABLE Treatment(

        TreatID         INT             IDENTITY(1,1),
        DoctorID        INT             FOREIGN KEY     REFERENCES
Doctor(DoctorID),
        PatID           INT             FOREIGN KEY     REFERENCES
Patient(PatID),
        Diagnose        NVARCHAR(150),
        prescription    NVARCHAR(150)   ,
        PRIMARY KEY(DoctorID, PatID, TreatID)
);
```

T 2.2 For the **1:N** relationship (Ward → Doctor), which table holds the **Foreign Key**? Why?

- ○ The Ward table
- ◉ The Doctor table
- ○ Both tables
- ○ A new junction table

T 2.3 The Treatment table has a **composite primary key**. What columns make up that composite PK, and why is a single column not sufficient?

The composite primary key consists of DoctorID, PatID, and TreatID. This is used because the table manages a many-to-many relationship, and the composite key ensures that every treatment record is uniquely and correctly linked to a specific doctor-patient pair for better data integrity.

Part 3 — ALTER TABLE

3 tasks · Fix structural mistakes using ALTER

T 3.1 You realize you forgot to add an **Email** column to the Doctor table. Write the ALTER TABLE statement to add it (`VARCHAR(150)`).

YOUR SQL

```
ALTER TABLE Doctor
ADD DocEmail    VARCHAR(150);
```

T 3.2 The Ward table's **Capacity** column was created as `VARCHAR(10)` by mistake. Write the SQL to change it to `INT`.

YOUR SQL

```
ALTER TABLE Ward
ALTER COLUMN    WardCapacity    INT;
```

T 3.3 Classify each statement as **DDL** or **DML**:

`ALTER TABLE Doctor ADD Email VARCHAR(150);`	DDL
`INSERT INTO Ward VALUES (1, 'ICU', 20);`	DML
`DROP TABLE Treatment;`	DDL
`UPDATE Doctor SET Specialization = 'Cardiology';`	DML

Part 4 — DML Operations

4 tasks · INSERT, UPDATE, DELETE, and cascading rules

T 4.1 Write INSERT statements to add the following data. Make sure to insert in the **correct order** (parent tables first).

WARD

WARDID	WARDNAME	CAPACITY
1	ICU	20
2	Pediatrics	30

DOCTOR

DOCID	DOCNAME	SPECIALIZATION	WARDID
101	Dr. Hassan	Cardiology	1
102	Dr. Nour	Pediatrics	2

PATIENT

PATID	PATNAME	DOB
501	Youssef	1990-05-12
502	Layla	2018-11-03

YOUR SQL — INSERT STATEMENTS (ALL 4 TABLES)

```
INSERT INTO Ward(WardID, WardName, WardCapacity)
VALUES
(1, 'ICU', 20),
(2, 'Pediatrics', 30);

INSERT INTO Doctor(DoctorID,DoctorName,DoctorSpecialization,WardID)
VALUES
(101, 'Dr. Hassan', 'Cardiology', 1),
(102, 'Dr. Nour', 'Pediatrics', 2);

INSERT INTO Patient(PatID,PatName,PatBirthDate)
VALUES
(501, 'Youssef', 1990-05-12),
(502, 'Layla', 2018-11-03);
```

T 4.2 Write an UPDATE statement to change Dr. Hassan's specialization from 'Cardiology' to **'Neurology'**.

YOUR SQL

```
UPDATE Doctor SET DoctorSpecialization = 'Neurology' WHERE DoctorID =
101;
```

T 4.3 What happens if you run `UPDATE Doctor SET WardID = 1;` **without a WHERE clause**?

If you run that command without a WHERE clause, it will update every single row in the Doctor table, setting everyone's WardID to 1. This is a common mistake that leads to massive data loss because you lose the original ward assignments for all doctors simultaneously.

T 4.4 If the Doctor→Ward FK is defined with **ON DELETE CASCADE**, what happens when we run `DELETE FROM Ward WHERE WardID = 1;`?

- ○ The DELETE is blocked with an error
- ◉ Ward 1 is deleted AND all doctors in Ward 1 are deleted
- ○ Ward 1 is deleted, doctors' WardID becomes NULL
- ○ Only the ward is deleted, doctors keep WardID = 1

Part 5 — SELECT First, Then JOINs

4 tasks · Single-table SELECT recap and high-level JOIN concepts

---

T 5.1 Write a **single-table SELECT** to list doctor name and specialization from the **Doctor** table only.

YOUR SQL

```
SELECT DoctorName, DoctorSpecialization FROM Doctor;
```

T 5.2 Suppose you run: `SELECT DocName, WardID FROM Doctor;` Why may this be less readable for a report than showing ward names?

YOUR EXPLANATION

This query is less readable because it displays the WardID (a numeric ID) instead of the WardName. Users usually don't know which ID belongs to which department, so showing "ICU" or "Pediatrics" is much more intuitive than showing "1" or "2" in a report.

T 5.3 Fill in the conceptual JOIN link between Doctor and Ward: `Doctor.___ = Ward.___`

YOUR ANSWER

```
Doctor.WardID = Ward.WardID
```

T 5.4 In one or two lines: what does JOIN do at a high level in a normalized database?

At a high level, a JOIN combines rows from two or more tables based on a related column between them. It allows you to reconstruct the relationships in a normalized database to present data as a single, unified result.